

Phase 10: RAG Chain

Hospital Patient Helpdesk Chatbot

Python module: 06_rag_pipeline/10_rag_chain.py **Jupyter notebook:**

13_notebooks/10_rag_chain.ipynb **Purpose:** Combine retrieval, safe prompt construction, and provider-neutral generation to produce final chatbot answers with citations and source provenance.

1. Phase objective

Phase 10 is the orchestration layer of the RAG workflow. It does not reimplement retrieval, prompts, or model access. Instead, it coordinates the public interfaces from:

1. Phase 7: retrieve and rank approved hospital evidence.
2. Phase 8: select the correct safety mode and construct a citation-ready prompt.
3. Phase 9: generate and validate a provider-neutral answer.

The final result is a stable answer contract for the future FastAPI backend and Streamlit interface.

2. Folder placement

```
hospital_patient_helpdesk_chatbot/
|-- 01_data/
|   |-- sample_queries/
|   |   |-- test_questions.csv
|   |   |-- processed/
|   |       |-- 10_rag_answers.json
|   |       |-- 10_rag_report.json
|   |       |-- 10_rag_audit.csv
|   |       |-- 10_failed_rag_answers.json
|   |       |-- plots/
|   |           |-- 10_rag_stage_latency.png
|   |           |-- 10_rag_answer_modes.png
|-- 02_config/
|   |-- prompt_config.yaml
|-- 05_vector_store/
|   |-- chroma_db/
|   |   |-- 06_vector_index.sqlite3
|-- 06_rag_pipeline/
|   |-- 07_retriever.py
|   |-- 08_prompt_template.py
|   |-- 09_llm_client.py
|   |-- 10_rag_chain.py
|   |-- 10_rag_chain.md
|   |-- 10_rag_chain.pdf
|-- 13_notebooks/
|   |-- 10_rag_chain.ipynb
|-- .env
|-- README.md
```

Every artifact generated by this phase begins with 10_.

3. Input files

01_data/sample_queries/test_questions.csv

Contains synthetic helpdesk questions used for end-to-end validation. The question column is required. Existing category, expected-source, and safety-class columns remain useful for evaluation context.

05_vector_store/chroma_db/06_vector_index.sqlite3

The Phase 6 vector index stores approved chunks, float32 embeddings, and retrieval metadata. Phase 7 validates its model, dimension, backend, schema version, and database integrity before querying it.

02_config/prompt_config.yaml

Defines the base hospital assistant instruction and the approved insufficient-context message. Phase 8 adds source delimiters, prompt-injection defenses, citation rules, emergency routing, and unsafe-medical-advice refusal instructions.

.env

Controls the Phase 9 provider and model. The default is `offline`, which makes the included validation deterministic and private. Hosted credentials are never placed in Phase 10 artifacts.

4. Output files

Numbered artifact	Description
10_rag_answers.json	Final question/answer records with citations, sources, confidence, safety labels, provider information, and stage timings.
10_rag_report.json	Aggregate counts, configuration, mode and confidence distributions, latency summary, and artifact inventory.
10_rag_audit.csv	One compact operational row for each successful answer.
10_failed_rag_answers.json	Stable answer ID, failing stage, and sanitized error for each unsuccessful question.
10_rag_stage_latency.png	Stacked retrieval, prompt, and generation latency per question.
10_rag_answer_modes.png	Counts of grounded, emergency, insufficient-context, and unsafe-medical-advice answers.

The plots are operational diagnostics. They do not establish medical safety or factual correctness.

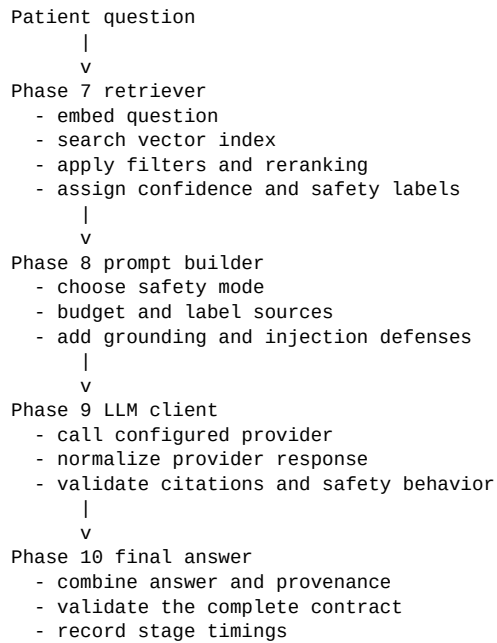
5. Final answer schema

Each item in `10_rag_answers.json` follows this contract:

```
{
  "answer_id": "ANSWER-001",
  "chain_version": "1.0",
  "question": "How can I book an appointment?",
  "answer": "Use the patient portal ... [S1]",
  "mode": "grounded_answer",
  "citations": ["[S1]"],
  "sources": [
    {
      "citation": "[S1]",
      "chunk_id": "...",
      "source_file": "faqs/hospital_faqs.json",
      "source_type": "json",
      "department": "Portal Support",
      "content_category": "appointments",
      "page_reference": null,
      "score": 0.74580427
    }
  ],
  "retrieval_confidence": "high",
  "safety_labels": [],
  "provider": "offline",
  "model": "offline-grounded-v1",
  "finish_reason": "stop",
  "timings": {
    "retrieval_ms": 4.0,
    "prompt_ms": 0.5,
    "generation_ms": 0.5,
    "total_ms": 5.0
  }
}
```

`citations` contains labels actually used in the answer. `sources` contains the approved evidence supplied to the model, including sources that were available but not cited in the final text.

6. End-to-end execution flow



7. Python module code sections

Chain configuration

RAGChainConfig validates the retrieval count, candidate pool, hybrid vector weight, prompt source budget, context character budget, and minimum confidence. `build_stage_configs()` translates those values into the native Phase 7 and Phase 8 configuration objects.

Final response contracts

AnswerSource, StageTimings, and RAGAnswer define the data returned to later API and UI phases. ChainRunResult identifies all generated artifacts and run totals.

Dependency loading

`import_module()` safely imports numbered Python files by path and registers them in `sys.modules`, which supports their dataclasses. `load_dependencies()` loads Phases 7, 8, and 9 from the project folder.

Source mapping

`source_records()` converts Phase 8 prompt sources into the public Phase 10 provenance schema. It preserves the citation label, chunk ID, source file, source type, department, category, optional page reference, and ranking score.

One-question orchestration

`answer_question()` is the central chain function. It:

1. normalizes and validates the question;
2. retrieves evidence through Phase 7;
3. constructs and validates the prompt through Phase 8;
4. creates the configured provider through Phase 9;
5. generates and validates the LLM response;
6. combines answer text and source provenance; and

7. validates the complete final contract.

Retrieval, prompt, generation, and total time are measured separately.

Final validation

`validate_chain_answer()` rejects empty questions or answers, unknown citations, uncited grounded answers, missing emergency routing, missing unsafe-advice refusal, and negative timing values.

Batch evaluation

`run_chain_evaluation()` loads shared dependencies once, processes every sample question independently, writes successful answers and audit rows, captures sanitized failures, creates plots, and writes the aggregate report.

Per-question isolation prevents one bad record or temporary provider failure from discarding successful answers.

Diagnostics

`generate_plots()` creates:

- a stacked latency plot that separates retrieval, prompt, and generation cost; and
- an answer-mode plot that confirms grounding and safety routes.

Command-line interface

Run the full offline sample evaluation:

```
python 06_rag_pipeline/10_rag_chain.py
```

Run one question:

```
python 06_rag_pipeline/10_rag_chain.py --question "Where is cardiology?"
```

Use a configured provider:

```
python 06_rag_pipeline/10_rag_chain.py --provider openai
```

Review privacy authorization and provider cost before using hosted models.

8. Notebook code sections

The notebook provides a readable, executable walkthrough:

1. Resolves the project path from the workspace root, project root, or notebook folder.
2. Imports the shared Phase 10 Python module.
3. Loads Phases 7, 8, and 9 through the chain.
4. Configures deterministic offline generation.
5. Traces one appointment question through the final answer contract.
6. Demonstrates emergency and unsafe-medical-advice routing.
7. Runs all 12 synthetic sample questions.
8. Validates citations, output counts, failures, and mode distribution.
9. Displays both numbered diagnostic plots.

9. Notebook versus Python module

Topic	10_rag_chain.ipynb	10_rag_chain.py
Primary role	Interactive learning, trace inspection, safety demonstration, and visual validation.	Reusable orchestration for API, UI, tests, and automation.

Topic	10_rag_chain.ipynb	10_rag_chain.py
Business logic	Imports and calls the module.	Owns chain configuration, orchestration, validation, batching, artifacts, and CLI.
Typical execution	Run cells in order.	Import <code>answer_question()</code> or run from the command line.
Provider in examples	Explicitly forced to offline.	Uses <code>.env</code> or <code>--provider</code> .
Outputs	Displays representative records, reports, and plots inline.	Writes numbered JSON, CSV, and PNG artifacts.
Failure behavior	Assertions stop the visible walkthrough.	Isolates and records failures per question during batch execution.

The notebook and module stay compatible because the notebook does not duplicate chain logic.

10. Validation results

The included offline run completed:

- 12 input questions;
- 12 final answers;
- 0 failed answers;
- 9 grounded answers;
- 1 emergency response;
- 2 unsafe-medical-advice refusals;
- 5 high-confidence retrievals; and
- 7 medium-confidence retrievals.

All answer citations belong to the source list for their own question. No hosted API call is made during this validation.

11. Healthcare safety and privacy

- The chain must not diagnose disease, prescribe treatment, or recommend medication dosage.
- Possible emergencies must be directed to local emergency services immediately.
- Answers must remain grounded in approved hospital evidence and preserve traceable citations.
- Retrieval confidence is an information-retrieval signal, not medical certainty.
- Protected health information must not be sent to external providers without appropriate authorization, contracts, controls, and necessity.
- Logs and failure records should minimize patient text and never contain API keys.
- Phase 11 guardrails, evaluation, monitoring, and human governance remain necessary before deployment.

Phase 10 provides an end-to-end hospital information workflow. It is not clinical decision support and does not replace qualified healthcare professionals.